gems/activerecord-2.3.2/lib/active_record/validations.rb

```
module ActiveRecord
  module Validations
    # ...

    def self.included(base)
      base.extend ClassMethods
      # ...
    end

    module ClassMethods
      def validates_length_of(*attrs) # ...
      # ...
    end

    def valid?
      # ...
    end

    # ...
  end
end
```

Does the code above look familiar? You've already seen this technique in *The VCR Example*, on page 159. Here's a quick recap. When ActiveRecord::Base includes Validations, three things happen:

1. The instance methods of Validations, such as valid?, become instance methods of Base. This is just regular module inclusion.

2. Ruby calls the included *Hook Method (157)* on Validations, passing ActiveRecord::Base as an argument. (The argument of included is also called base, but that name has nothing to do with the Base class—instead, it comes from the fact that a module's includer is sometimes called "the base class.")

3. The hook extends Base with the ActiveRecord::Validations::ClassMethods module. This is a *Class Extensions (130)*, so the methods in ClassMethods become class methods on Base.

As a result, Base gets both instance methods like valid? and class methods like validates_length_of.

This idiom is so specific that I hesitate to call it a spell. I'll refer to it as the *include-and-extend* trick. VCR borrowed it from Rails, as did many other Ruby projects throughout the years. Include-and-extend gives you a powerful way to structure a library: each module contains a well-isolated piece of functionality that you can roll into your classes with a simple include. That functionality can be implemented with instance methods, class methods, or both.